

Improving Code Quality 2

Formative Activity

Run the code against a variety of linters to test the code quality:

- `pylint code_with_lint.py`
- `pyflakes code_with_lint.py`
- `pycodestyle code_with_lint.py`
- `pydocstyle code_with_lint.py`

Compare the effectiveness of each tool in defining and identifying code quality. What can you conclude about the effectiveness of each approach?

Code

This formative activity used the `code_with_lint.py` code referred to in Unit 10. The code can be found in Appendix 1. This was improved using a range of linters.

Pylint

Pylint revealed the following errors:

Error
Unused modules
Unused functions from modules
from Module import * - * not allowed
Global Variables must be capitalised
Variables can not be redefined
Unreachable code
Unnecessary parentheses
Unnecessary semi-colon
Logical convention is not None rather then != None
No trailing whitespace
Must include module docstrings
Must include class docstrings
Must include function docstrings
Returns must be consistent
Highlight useless returns
Must include at least 2 public methods for a class
Highlights unused variables
Must include final newline statement
Code quality score

Pyflakes

Pyflakes returned fewer issues with the code quality, as summarised in the table below:

Error	Pylint	Pyflakes
Unused modules	X	X
Unused functions from modules	X	X
from Module import * - * not allowed	X	X
Global Variables must be capitalised	X	
Variables can not be redefined	X	
Unreachable code	X	
Unnecessary parentheses	X	
Unnecessary semi-colon	X	
Logical convention is not None rather than != None	X	
No trailing whitespace	X	
Must include module docstrings	X	
Must include class docstrings	X	
Must include function docstrings	X	
Returns must be consistent	X	
Highlight useless returns	X	
Must include at least 2 public methods for a class	X	
Highlights unused variables	X	X
Must include final newline statement	X	
Code quality score	X	

Pycodestyle

Pycodestyle returned fewer issues with the code style. Some of these were not highlighted by pylint as summarised in the table below:

Error	Pylint	Pyflakes	Pycodestyle
Unused modules	X	X	
Unused functions from modules	X	X	
from Module import * - * not allowed	X	X	
Global Variables must be capitalised	X		
Variables can not be redefined	X		
Unreachable code	X		

Error	Pylint	Pyflakes	Pycodestyle
Unnecessary parentheses	X		
Unnecessary semi-colon	X		X
Logical convention is not None rather than != None	X		X
No trailing whitespace	X		X
2 blank lines between functions			X
1 whitespace needed around operators			X
No whitespace after (			X
Whitespace after keyword			X
Lines too long			Max 79 Characters
Must include module docstrings	X		
Must include class docstrings	X		
Must include function docstrings	X		
No space for default between '=' for default parameters			X
Returns must be consistent	X		
Highlight useless returns	X		
Must include at least 2 public methods for a class	X		
Highlights unused variables	X	X	
Must include final newline statement	X		X
Code quality score	X		

Pydocestyle

Pydocestyle returned fewer issues with the code style. Some of these were not highlighted by pylint as summarised in the table below:

Error	Pylint	Pyflakes	Pycodestyle	Pydocstyle
Unused modules	X	X		
Unused functions from modules	X	X		
from Module import * - * not allowed	X	X		
Global Variables must be capitalised	X			
Variables can not be redefined	X			
Unreachable code	X			
Unnecessary parentheses	X			
Unnecessary semi-colon	X		X	
Logical convention is not None rather than != None	X		X	
No trailing whitespace	X		X	
2 blank lines between functions			X	
1 whitespace needed around operators			X	
No whitespace after (			X	
Whitespace after keyword			X	
Lines too long			Max 79 Characters	
Must include module docstrings	X			X
Must include class docstrings	X			
Must include function docstrings	X			
One line docstrings should fit on one line				X
Docstrings must end with ‘				X
Docstrings must start with an imperative				X

Error	Pylint	Pyflakes	Pycodestyle	Pydocstyle
Multiline Docstrings must have a blank line between the summary and the description				X
No space for default between '=' for default parameters			X	
Returns must be consistent	X			
Highlight useless returns	X			
Must include at least 2 public methods for a class	X			
Highlights unused variables	X	X		
Must include final newline statement	X		X	
Code quality score	X			

Summary

This shows the need to use a range of linters to improve code, both in terms of functionality and readability. One linter is not sufficient as each has separate rules. When considering improving code, it is important to pick the right tool for the task. For example, pydocstyle focuses in depth on the rules for the overall document layout such as formatting of docstrings. In contrast pycodestyle focuses on the readability of code, while pylint considers the functionality of the code itself.

Code will be improved when running a range of linters and this will in turn aid both collaboration between developers and ensure maintainability.

References

Anaya, M. (2021) *Clean Code in Python: Develop maintainable and efficient code*. 2nd edn. Birmingham: Packt Publishing.

Appendix 1 – code_with_liny.py

```
import io
from math import *

from time import time

some_global_var = 'GLOBAL VAR NAMES SHOULD BE IN ALL_CAPS_WITH_UNDERSCORES'

def multiply(x, y):
    """
    This returns the result of a multiplication of the inputs
    """
    some_global_var = 'this is actually a local variable...'
    result = x* y
    return result
    if result == 777:
        print("jackpot!")

def is_sum_lucky(x, y):
    """This returns a string describing whether or not the sum of input is
    lucky
    This function first makes sure the inputs are valid and then calculates
    the
    sum. Then, it will determine a message to return based on whether or not
    that sum should be considered "lucky"
    """
    if x != None:
        if y is not None:
            result = x+y;
            if result == 7:
                return 'a lucky number!'
            else:
                return( 'an unlucky number!')

        return ('just a normal number')

class SomeClass:

    def __init__(self, some_arg, some_other_arg, verbose = False):
        self.some_other_arg = some_other_arg
        self.some_arg = some_arg
        list_comprehension = [(100/value)*pi) for value in some_arg if value
        != 0]

        time = time()
        from datetime import datetime
        date_and_time = datetime.now()
        return
```